

Owner A 新手闭卷指南：GQA6、Attention 路由与 GDN

目标：不背完整 vLLM，不熟悉 Triton 也能依靠官方源码，重新完成 Owner A 的三项工作：GQA6 prefill、宿主路由、GDN RMSNorm+SiLU。本文只解释正确现状，不要求当前机器构建或跑卡。

0. 一页总图

```
Qwen attention
-> official ROCm AITER backend.forward
    -> exact common gate
        |
        |--- miss -> official unified_attention
        |--- hit  -> B page784.prefill
            |
            |--- True  -> output complete
            |--- False -> A GQA6 -> output complete

Qwen GatedDeltaNet Part 3
-> qwen35_gdn_rmsnorm(norm, x, z)
    |
    |--- target layout + gfx936 -> A Triton RMSNorm+SiLU
    |--- otherwise               -> official norm reshape fallback
```

闭卷只背两个原则：

- 1. 路由必须窄门、互斥、可回退。
- 2. kernel 只特化调度，不改变官方数学。

1. 先分清 A 的三个任务

任务	文件	对上接口	自定义计算
GQA6	rocm_aiter_unified_attention_gqa6.py	prefill(...)->None	Triton online attention
路由	rocm_aiter_unified_attn.py	官方 backend forward()	无新数学，只选择路径

任务	文件	对上接口	自定义计算
GDN	<code>gfx936.py</code> + <code>qwen3_5.py</code>	<code>qwen35_gdn_rmsnorm(...)</code>	Triton RMSNorm × SiLU

A 不负责 B 的 page784 内部，也不负责 C 的 K5120 HIP kernel。

2. 官方积木：现场先找，不从空白背

2.1 搜索地图

```
# 官方 attention host 和 fallback 调用
rg -n "class RocmAiterUnifiedAttentionImpl|def forward" \
  vllm/v1/attention/backends/rocm_aiter_unified_attn.py

# 官方 paged attention: block table、stride、online softmax
rg -n "physical_block_idx|stride_k_cache|max_score|exp_sum|tl.dot" \
  vllm/v1/attention/ops/triton_unified_attention.py

# 官方 RMSNormGated 数学和 fallback
rg -n "class RMSNormGated|forward_native|forward_cuda" \
  vllm/model_executor/layers/layernorm.py

# GDN Part 3 插入位置
rg -n "core_attn_out|self.norm" vllm/model_executor/models/qwen3_5.py
```

对应 vLLM v0.18.1 官方入口：

- https://github.com/vllm-project/vllm/blob/v0.18.1/vllm/v1/attention/backends/rocm_aiter_unified_attn.py
- https://github.com/vllm-project/vllm/blob/v0.18.1/vllm/v1/attention/ops/triton_unified_attention.py
- https://github.com/vllm-project/vllm/blob/v0.18.1/vllm/model_executor/layers/layernorm.py
- https://github.com/vllm-project/vllm/blob/v0.18.1/vllm/model_executor/models/qwen3_5.py

2.2 什么直接借，什么必须自己理解

FIND IN OFFICIAL	OWNER A MUST DERIVE		
metadata/block-table meaning	Q24/KV4 means 6 Q heads per KV head		
runtime stride addressing	pair 6 Q heads as 3 x 2		
online-softmax recurrence	grid and row mapping		
official fallback call	exact common gate and route order		
RMSNormGated formula	fixed [T,48,128] row mapping		

3. GQA6 是什么

3.1 GQA 6:1

目标配置：24 个 query heads、4 个 KV heads、head dimension 256。

Q head axis
0 1 2 3 4 5 | 6 7 8 9 10 11 | ... | 18 19 20 21 22 23
└───────── KV head 0 ─────────┘ └───────── KV head 1 ─────────┘ └───────── KV head 3 ─────────┘

Within one KV group:
pair 0 = Q[0,1] pair 1 = Q[2,3] pair 2 = Q[4,5]

一个 CTA 同时算共享同一 K/V tile 的两个 Q head，从而减少重复加载 K/V。

3.2 grid 如何映射

```
grid = (ceil_div(query_len, BLOCK_ROWS/2), 12, 1)
```

第二维 12 来自 4 KV heads * 3 Q-head pairs。

Tensor: query rows handled by one program
 Formula: $\text{token} = \text{query_block} * (\text{BLOCK_ROWS} / 2) + \text{row} // 2$
 $\text{q_head} = \text{kv_head} * 6 + \text{pair} * 2 + \text{row} \% 2$



BLOCK_ROWS=16 时一个 program 覆盖 8 个 query token；BLOCK_ROWS=64 时覆盖 32 个。

3.3 Q 的地址

query 连续布局 [Q,24,256]，所以：

```
offset = token * (24*256) + q_head * 256 + dimension
        = token * 6144   + q_head * 256 + dimension
```

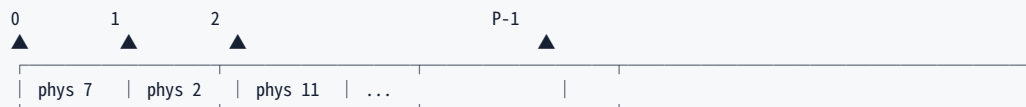
这个硬编码只安全，因为宿主 gate 已保证 query/output 连续且形状固定。K/V cache 可能是交错 view，必须使用各自 runtime stride，不能照抄连续公式。

4. GQA6 的 paged KV 地址

4.1 logical token 到 physical page

```
logical_position -> logical_page=position//784
                  -> physical_page=block_table[logical_page]
                  -> position_in_page=position%784
```

Tensor: block_table, logical page axis 0..P-1
 Formula: $\text{physical_page}[p] = \text{block_table}[p]$



K 地址逐轴相加：

```
physical_page*Ks0 + position*Ks1 + kv_head*Ks2 + dim*Ks3
```

V 用自己的 `Vs0..Vs3`。这段模式直接对照官方 `triton_unified_attention.py` 的 `physical_block_idx` 与 `stride_k_cache_*` 写。

4.2 tile 跨 784 page 边界

一个 KV tile 可能从 page 尾跨到下一页：



代码先求 `first_page_tokens=784-first_position`，column 小于它时用第一页，否则用下一页并从 position 0 重新计数。闭卷时必须画这张图，否则很容易只处理页内 tile。

5. Online softmax: GQA6 的数学核心

5.1 为什么不能保存全部 score

context 可很长。kernel 分 tile 读 K/V，同时保持每个 query row 的三个状态：

```
m = seen scores maximum
l = sum exp(score-m)
a = sum exp(score-m)*V
```

最终 `output=a/l`。

5.2 新 tile 到来时如何更新

旧最大值为 `m_old`，新 tile 最大值为 `tile_max`：

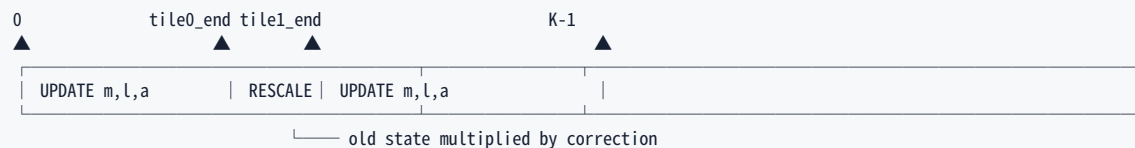
```

m_new = max(m_old, tile_max)
c      = exp(m_old-m_new)
p      = exp(scores_new-m_new)
a_new  = a_old*c + p @ V_new
l_new  = l_old*c + sum(p)

```

为什么要乘 c ? 旧状态以 m_old 为基准, 最大值变化后必须换到 m_new 的标尺。

Tensor: score stream for one (token,q_head), K axis 0..K-1
 Formula: each tile updates the same (m,l,a) state



闭卷不要背变量名, 只背顺序:

new max -> correct old numerator/denominator -> add new probabilities and $P@V$

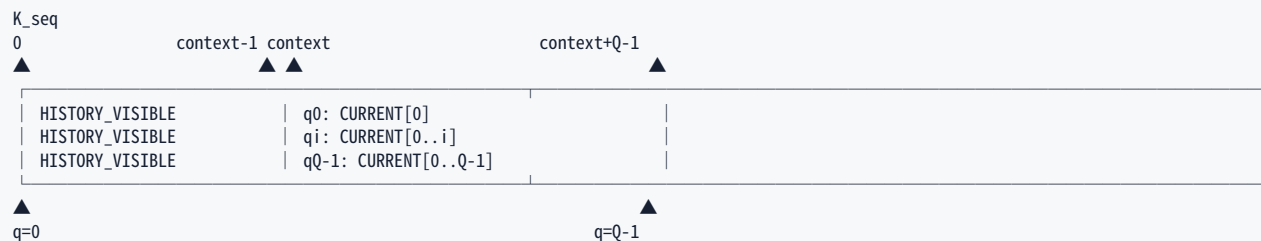
5.3 causal 上界

已有历史为 `context_len`。本轮 local query token i 可看:

history $0..context_len-1$ + current $0..i$

因此 key logical position 必须满足:

$key_position < context_len + i + 1$



6. 两套 tile 为什么存在

query length	BLOCK_ROWS	token/program	KV subtile	stages
<128	16	8	16	2
>=128	64	32	32×2	1

短 query 用小 CTA 避免浪费；长 query 用宽 CTA 提高 KV 复用，但把 64 KV token 分成两个 32-token subtile以适配目标矩阵指令/资源。这里是性能特化，online-softmax 数学不变。

7. Attention 宿主路由

7.1 构造期不变量

`__init__` 缓存不随 token 改变的条件：24/4/256、auto cache dtype、decoder、无 ALiBi/ window/sink/softcap、gfx936。避免每层每 token 重复设备查询。

7.2 forward 运行期 gate

检查本次输入：prefill、单请求、page shape [784,4,256]、BF16、query/output contiguous、无 output scale。

Tensor/config gate

DEVICE + MODEL INVARIANTS: cached in <code>__init__</code>	
REQUEST + TENSOR CONDITIONS: checked in forward	
HIT -> page784(True return / False GQA6)	
MISS -> official unified_attention	

7.3 三条互斥路径

必须能口述：

- page784 True：它已写完整 output，立即返回。

- page784 False: 它承诺没写 output, 运行 GQA6。
- common gate miss: 不要进入 B/A kernel, 保留官方 AITER 全功能 fallback。

不能让 GQA6 自己重复公共 gate; 唯一真相应在宿主, 避免 gate 漂移。

8. GDN RMSNorm+SiLU

8.1 官方公式

输入 `x=core_attn_out`, gate 为 `z`, 每行 Hidden=128:

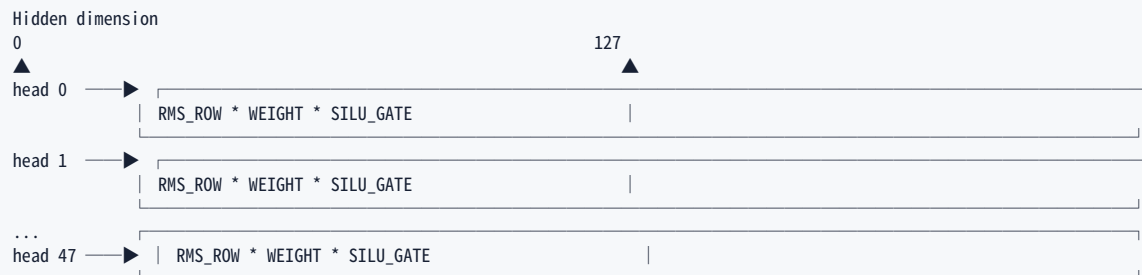
```
rms(x) = x * rsqrt(mean(x*x)+eps)
silu(z)= z * sigmoid(z)
y      = rms(x) * weight * silu(z)
```

所有中间计算转 FP32, 最后存回 `x dtype`。

8.2 张量 layout

`x` 为 `[T,48,128]`; gate 的 stride 是 `(16384,128,1)`, 说明每个 token 在底层 `z storage` 跨 16384 元素, 而目标 48 个 head 每个宽 128。

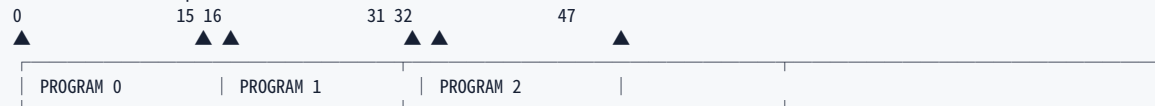
```
Tensor: x and selected z region, shape=[T,48,128]
Formula: y[t,h,d]=norm(x[t,h,:])[d]*w[d]*silu(z[t,h,d])
```



8.3 grid 为什么是 T*3

一个 program 处理 16 行；每 token 有 48 行，所以 $48/16=3$ 个 program。总 grid 为 T*3。

Flattened row axis per token



8.4 gate offset

flattened row `r` 对应：

```
token = r // 48
head = r % 48
offset = token*16384 + head*128 + dim
```

不要用 `x` 的连续 offset 读取 gate；两者 layout 不同。

8.5 fallback 是正确性的一部分

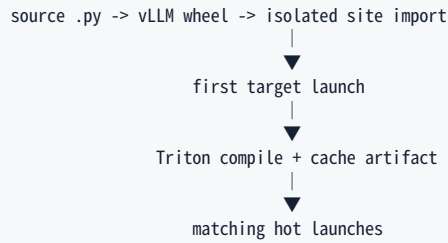
非目标 layout、CPU、非 gfx936 必须执行：

```
norm(x.reshape(-1,128), gate.reshape(-1,128)).reshape_as(x)
```

fast path 只替换 Qwen GDN Part 3 的 epilogue，不复制投影、recurrent core、state 或输出投影。

9. 构建与运行时生效

A 的修改全是 Python/Triton：wheel 构建把 Python DSL 打包；目标输入第一次 launch 时 Triton JIT。A 不重编 FlashAttention，也没有单独 AOT kernel 命令。



原 Owner A 文档中的构建/测试命令是比赛设备上的已验证流程。本机设备不匹配时，只学习并保留这些命令，不运行全量构建或 GPU 实验。真正执行时要特别确认：临时 worktree 包含提交、PYTHONPATH 指向新 site、首次 JIT 不计性能。

目标比赛容器中的完整流程如下；这是操作手册，不代表本文编写环境执行过：

```
REPO_ROOT=/public/home/tangyu408/Qwen_DCU_Worker_0/pra2026-bh408-gqa-page784-k5120
BUILD_SOURCE=/tmp/qwen35-build-source
ARTIFACT_ROOT=/tmp/qwen35-build
SITE_DIR="$ARTIFACT_ROOT/site"
CACHE_ROOT="$ARTIFACT_ROOT/cache"

cd "$REPO_ROOT"
git worktree add --detach "$BUILD_SOURCE" HEAD
mkdir -p "$ARTIFACT_ROOT/{dist,bdist,cache}"
cd "$BUILD_SOURCE"
VLLM_TARGET_DEVICE=rocm MAX_JOBS=16 python3 setup.py \
  build --build-base "$ARTIFACT_ROOT/build" \
  bdist_wheel --bdist-dir "$ARTIFACT_ROOT/bdist" \
  --dist-dir "$ARTIFACT_ROOT/dist" \
python3 -m pip install --no-deps --target "$SITE_DIR" \
  "$ARTIFACT_ROOT/dist/vllm-*.whl

cd "$REPO_ROOT"
ruff check vllm/v1/attention/ops/rocm_aiter_unified_attention_gqa6.py \
  vllm/v1/attention/backends/rocm_aiter_unified_attn.py \
  vllm/model_executor/layers/fla/ops/gfx936.py \
  vllm/model_executor/models/qwen3_5.py
RUN_DIR="$(mktemp -d /tmp/qwen35-a-check.XXXXXX)"
(cd "$RUN_DIR" && HIP_VISIBLE_DEVICES=0 \
  TRITON_CACHE_DIR="$CACHE_ROOT/a-triton" PYTHONPATH="$SITE_DIR" \
  python3 "$REPO_ROOT/docs/cscv/verify_qwen35_optimizations.py" gqa6 gdn)
```

若 BUILD_SOURCE 已存在，不要盲目覆盖或删除；先确认它属于本次构建。还要确认 HEAD 已含待构建修改，否则 detached worktree 会构建旧提交。

10. 闭卷实现顺序

10.1 GQA6

1. 从官方 unified attention 找 block table、stride、online-softmax 模式。
2. 写固定接口，假设公共 gate 已证明 24/4/256 与连续 Q/O。
3. 推导 $\text{kv_head} = \text{query_group} // 3$ 、 $\text{pair} = \text{query_group} \% 3$ 。
4. 推导 row $\rightarrow$ token/head 映射。
5. 用 runtime stride 写跨页 K/V 地址。
6. 写 causal mask。
7. 按 $\text{max} \rightarrow \text{correction} \rightarrow \text{denominator} \rightarrow \text{P@V}$ 更新。
8. 除 normalizer，写 output。
9. host 按 query 长度选 tile/options/grid。

10.2 路由

1. 在官方 backend 保留原 fallback。
2. `__init__` 缓存设备/模型不变量。
3. `forward` 检查请求和 tensor 条件。
4. 命中后严格 `page784` $\rightarrow$ GQA6。
5. 三路都只写一次 output。

10.3 GDN

1. 从官方 `RMSNormGated.forward_native` 抄公式顺序。
 2. 只对 `[T, 48, 128]` 和目标 gate stride 开门。
 3. flatten 行，16 行/program。
 4. x、gate、weight 转 FP32。
 5. RMSNorm 后乘 weight，再乘 `SiLU(gate)`。
 6. 非目标调用传入的官方 norm。
-

11. 十条闭卷不变量

1. $24/4=6$ ，每个 KV head 对应三个 Q-head pair。
 2. program row 偶/奇对应同 token 的两个 Q head。
 3. K/V cache 使用各自完整 runtime stride。
 4. KV tile 跨 784 页时必须切换 block-table page。
 5. causal 上界为 `context+local_token+1`。
 6. online softmax 最大值变化后，旧 numerator/denominator 都乘 correction。
 7. page784 True 立即返回；False 才运行 GQA6。
 8. common gate miss 完整保留官方 AITER。
 9. GDN 数学是 $\text{FP32 RMSNorm} \times \text{weight} \times \text{SiLU}(z)$ 。
 10. GDN gate layout 不匹配必须走官方 fallback。
-

12. 从空白文件重建时的最小伪代码

下面故意不是当前源码的逐字复制。它用于提醒“先写数据关系，再对照官方 API 补语法”。

12.1 GQA6 host launcher 骨架

```
def prefill(query, key_cache, value_cache, output, metadata, scale):
    # Common gate already proved Q24/KV4/D256, BF16, single request, Q/O contiguous.
    q = query[:metadata.num_actual_tokens]
    out = output[:metadata.num_actual_tokens]

    long_query = len(q) >= 128
    block_rows = 64 if long_query else 16
    tokens_per_program = block_rows // 2
    grid_q = ceil_div(metadata.max_query_len, tokens_per_program)

    kernel[(grid_q, 4 * 3, 1)](
        out, q, key_cache, value_cache, metadata.block_table,
        metadata.max_query_len,
        metadata.max_seq_len - metadata.max_query_len,
        scale,
        CACHE_STRIDES=(key_cache.stride(), value_cache.stride()),
        BLOCK_ROWS=block_rows,
        # Add target compiler options after correctness structure is complete.
    )
```

现场需要从官方确认的不是上述数学，而是当前 Triton launch 参数名和 metadata 字段是否仍一致。

12.2 GQA6 kernel 骨架

```
@triton.jit
def kernel(..., CACHE_STRIDES: tl.constexpr, BLOCK_ROWS: tl.constexpr):
    q_block = tl.program_id(0)
    group = tl.program_id(1)
    kv_head = group // 3
    pair = group % 3

    row = tl.arange(0, BLOCK_ROWS)
    dim = tl.arange(0, 256)
    token = q_block * (BLOCK_ROWS // 2) + row // 2
    q_head = kv_head * 6 + pair * 2 + row % 2
    q = load_contiguous_query(token, q_head, dim)

    m = -inf_per_row
    l = initial_denominator_per_row
    acc = zero_fp32_rows_by_dim

    for each_kv_tile_needed_by_this_query_block:
        logical_position = tile_start + column
        physical_page, position = lookup_page_and_handle_784_boundary(...)
        k = load_with_k_runtime_strides(...)
        v = load_with_v_runtime_strides(...)
        score = scale * dot(q, k)
        score = where(key_position < context + token + 1, score, -inf)

        m_new = maximum(m, row_max(score))
        correction = exp(m - m_new)
        probability = exp(score - m_new)
        acc = acc * correction + probability @ v
        l = l * correction + row_sum(probability)
        m = m_new

    store(output, acc / l)
```

闭卷检查顺序：先检查 row→token/head，再检查 page/stride，再检查 causal，最后检查 online softmax。不要同时调四类错误。

12.3 路由骨架

```
def forward(...):
    # Keep all official preprocessing and feature branches.
    if exact_common_gate:
        if page784.prefill(...):
            return output
        gqa6.prefill(...)
        return output

    # Preserve the original official call and every original argument.
    self.unified_attention(...)
    return output
```

12.4 GDN 骨架

```
def qwen35_gdn_rmsnorm(norm, x, gate):
    if not exact_target_layout_and_device:
        return norm(x.reshape(-1, 128), gate.reshape(-1, 128)).reshape_as(x)

    output = empty_like(x)
    kernel[(x.shape[0] * 3,)](x, gate, norm.weight, output, norm.eps)
    return output

@triton.jit
def kernel(x, gate, weight, output, eps):
    rows = program_id * 16 + arange_16
    dims = arange_128
    x_fp32 = load_x(rows, dims).to(fp32)
    z_fp32 = load_gate_with_observed_stride(rows, dims).to(fp32)
    inv_rms = rsqrt(sum(x_fp32*x_fp32, axis=Hidden)/128 + eps)
    y = x_fp32 * inv_rms * weight_fp32 * (z_fp32 * sigmoid(z_fp32))
    store_output(rows, dims, y)
```

伪代码中的 helper 名称不是 Triton API；实现时应展开成指针 offset，并从当前官方源码复制 正确的 `tl.load/tl.store/tl.dot` 用法。

13. 静态审查与设备上验收清单

当前非目标设备只做前四项只读/静态审查；后续设备验收按原 Owner A 文档执行：

- 对照官方源确认 API/metadata 字段未漂移；
- 逐项检查 gate 与 fallback 未删除；
- 手算 row/head/page/stride 边界；

- 检查 kernel launch 前提由宿主 gate 覆盖；
- 目标设备验证 GQA6 的短尾、长 query、first prefill、交错 cache；
- 验证 page784 True/False/common miss 三路；
- GDN 验证 T=16/32/64/128/4096；
- GDN 验证非目标 stride fallback；
- 冷 cache 只证明 JIT，热 cache 才计性能；
- 最终从同一新 wheel 做服务级 DP1/DP2 冒烟。

14. 最常见错误

- 连续 cache 通过但交错 cache 错：输入地址偷用了连续 offset。
- page 末尾附近错：KV tile 跨 784 页逻辑缺失。
- 长 context 数值漂移：online-softmax correction 漏乘 numerator 或 denominator。
- 输出出现未来信息：causal 上界少/多了 1。
- page784 命中后又跑 GQA6：bool 契约理解错误。
- 非目标模型崩溃：公共 gate 过宽或官方 fallback 被改坏。
- GDN 部分 head 错：把 gate 当成与 x 相同 stride。
- GDN 小误差扩大：过早用 BF16 计算 RMS/SiLU。

15. 口述答案

A 在官方 ROCm AITER backend 中增加窄 gate。命中后先让 B 尝试 page784；B 返回 True 表示 output 完成，False 才运行 GQA6；公共 gate 不命中保留官方 unified attention。GQA6 针对 Q24/KV4/D256，把一个 KV head 的六个 Q head 配成三对，每个 CTA 复用 K/V tile 算两个 Q head。paged cache 用 block table 和 K/V 各自 runtime stride，跨 784 页时切页；每行用 FP32 online-softmax 的 max、normalizer、weighted value 状态，causal 上界是 context+local token+1。A 还在 Qwen GDN Part 3 对 [T,48,128] 融合官方 RMSNorm 和 SiLU gate，按 gate 的真实 stride 读取，非目标条件调用官方 norm。两类 kernel 都是 wheel 中的 Python/Triton DSL，在首次目标 launch 时 JIT。

16. 十轮自审记录

以下十轮均以 Owner A 原说明和当前源码为依据；只做文档与源码静态核对，未运行构建或设备 实验。

轮次	检查主题	检查结果与完善
1	职责覆盖	GQA6、宿主路由、GDN、A/B/C 边界均有独立章节
2	官方模板	补齐四个官方文件、符号搜索命令和“可借/需推导”边界
3	调用接口	明确 page784 bool 契约、GQA6 output 所有权和 GDN fallback
4	shape/grid	复算 $24/4=6$ 、12 groups、16/64 rows、GDN $T*3$
5	数学	逐项核对 causal 上界、online-softmax correction、RMSNorm $\times$ SiLU
6	地址/layout	补齐跨 784 页、K/V 独立 stride、gate stride 图
7	fallback/gate	检查 common miss、page784 False、非目标 GDN 三种回退
8	构建/JIT	补齐目标容器 wheel/隔离安装命令，区分 wheel 与首次 JIT
9	闭卷/验收	压缩为十条不变量，并映射短尾、长 query、交错 cache、GDN shapes
10	可视化/新手性	检查张量轴起止、平直边框、公式标签、逐步口述答案与常见错误

最终静态结论：本文可独立指导实现与构建；设备相关正确性和性能结论仍必须在原 Owner A 指定的 gfx936 比赛环境中验收，本文不把静态审查表述成实测。